Module 5 Tangent: Type Systems

Subtyping

If it walks like a duck, and it talks like a duck…
…then it probably won't explode at runtime?

Subtypes

“Language” Definitions

Subtyping is a relationship that allows a type to be used in place of a another.

- A type is a **subtype** if a value of that type can be used as the **supertype**.
  - e.g. assignment value
- In most languages, subtyping is determined based on inheritance and allows **polymorphism**.

Formal Requirements

A **type** is a classification over a <u>set of values</u> that share some form of behavior/properties.

- A type is a **subtype** if and only if all <u>provable properties</u> of the supertype still hold (LSP).
  - *Observed* behavior is not sufficient!
- Consequently, compile-time subtypes are a compromise around what can be proven.
  - e.g. List.add() can fail if immutable
  - int values must be valid list indices

Subtype Operations

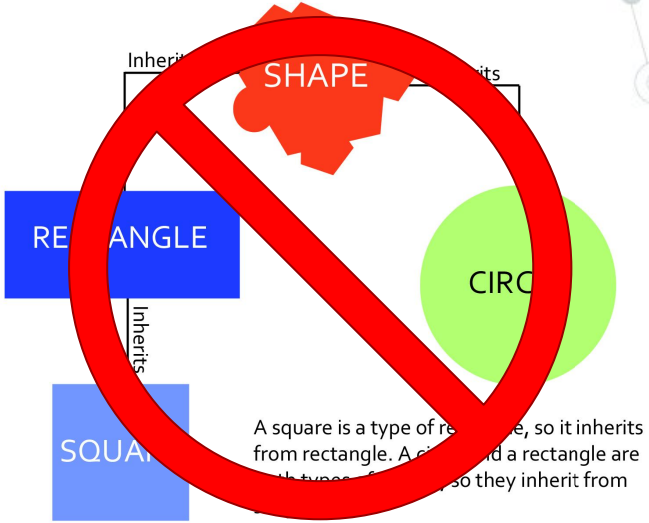
	String	**Integer**	**Unsigned Integer**
+	String (concat)	Integer (add)	U. Integer (add)
-	N/A	Integer (sub)	Integer (sub)
*****	String (repeat)	Integer (mult)	U. Integer (mult)
/	N/A	Integer (truncate) Double (division) *or DivideByZero	U. Integer (truncate) U. Double (division) *or DivideByZero

Liskov Substitution (LSP)

Subtypes Must be Substitutable

LSP states that if is S is a subtype of T, then all <u>provable</u> properties of T should hold true for S.

- The contract of S must be compatible with T
  - Preconditions can be weaked
  - Postconditions can be strengthened
  - Invariants must be maintained
- Mathematically, all Squares are Rectangles!
  - But programmatically, it's more nuanced
  - What happens if Rectangle is mutable?



Polymorphism

"Multiple Forms"

Polymorphism allows code (methods) to be used generically with multiple objects, each with their own definition.

- Methods use dynamic dispatch (i.e. runtime types) for the receiver.
  - Aka, the object's class.
- Polymorphism is a key part of most modern languages / software
  - Enables many abstractions

Styles of Polymorphism

Languages have varying approaches for polymorphism (or even multiple!).

- Subtype Polymorphism
  - Nominal subtyping, e.g. class inheritance or interfaces/traits
- Ad-hoc Polymorphism
  - Structural subtyping, e.g. duck typing & dynamic/runtime behavior
- Parametric Polymorphism (Generics)
  - Polymorphism over types, e.g. higher order functions.

Structural Subtyping is based on *structure*.

Types are subtypes because they *are**,
not because you *say* the are.

**at least at compile time.*

```java
//nominal subtyping: Java
Animal animal;

animal = new Dog();
animal.makeSound(); //Woof!
animal = new Cat();
animal.makeSound(); //Meow!

class Car {
  public void makeSound() {
    System.out.println("Honk!");
  }
}

animal = new Car(); //Not a subtype!
```

```typescript
//structural subtyping: TypeScript
let animal: Animal;

animal = new Dog();
animal.makeSound(); //Woof!
animal = new Cat();
animal.makeSound(); //Meow!

class Car {
  makeSound(): void {
    console.log("Honk!");
  }
}

animal = new Car();
animal.makeSound(); //Honk!
```

Nominal vs Structural Subtyping

Module 5 Tangent: Type Systems

TypeScript Fundamentals

An introduction to structural subtyping.

Slides Carnival

Free templates for all your presentation needs

For PowerPoint and Google Slides

100% free for personal or commercial use

Ready to use, professional and customizable

Blow your audience away with attractive visuals